POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Navier-Stokes equations for Laminar Flow Around a Cylinder

Report on Numerical Methods For PDE Project
High Performance Computing Engineering

Daniele Cursano, Marco Melzi, Lorenzo Terzi,

Academic year:
2025-2026

Abstract: This work presents a computational framework designed to re-evaluate and extend the canonical Schäfer–Turek (DFG) benchmark for laminar flow past a circular cylinder, serving as a baseline for the numerical analysis of the incompressible Navier–Stokes equations. The study encompasses two- and three-dimensional, steady and unsteady flow regimes, with quantitative assessment relying on integrated aerodynamic coefficients and point-wise pressure differentials. Extensive numerical experiments were conducted across a range of Reynolds numbers to evaluate the solver. A comprehensive benchmarking analysis of various preconditioning strategies was performed to assess their impact on computational efficiency, stability, and scalability.

1. Introduction

The Navier-Stokes equations are a fundamental set of partial differential equations in fluid mechanics that describe how the velocity and pressure fields of a fluid evolve over time. They serve as a mathematical representation of momentum balance for Newtonian fluids, incorporating the principles of mass conservation.
This project focuses on the numerical solution of the unsteady, incompressible Navier-Stokes equations by means of the Finite Element Method (FEM). The primary objective is to simulate the standard 2D and 3D benchmark problems of laminar flow past a cylinder. The simulations are conducted to analyze the fluid behavior under different flow regimes, specifically targeting Reynolds numbers up to $Re \leq 200$.
A crucial aspect of this study is the accurate computation of the aerodynamic forces exerted by the fluid on the obstacle. The project requires calculating and plotting the drag coefficient (C_D) and the lift coefficient (C_L) over time. These dimensionless coefficients are defined as:

$$C_D = \frac{F_D}{U^2 L}, \quad C_L = \frac{F_L}{U^2 L} \tag{1}$$

where F_D and F_L denote the forces acting on the cylinder parallel and perpendicular to the flow direction, respectively, U is the reference flow velocity, and L is the cross-sectional length of the obstacle.
Particular attention is given to the linear algebra framework implemented via the `deal.II` and Trilinos libraries, including the implementation of specialized block preconditioners necessary to efficiently resolve the saddle-point nature of the discretized Navier-Stokes system. As we have seen in class during the course.

2. Mathematical Model

2.1. Strong Formulation

The physical problem is governed by the non-stationary Navier-Stokes equations for an incompressible viscous fluid. Let $\Omega \subset R^d$ (with $d = 2, 3$) be the computational domain and $\partial\Omega$ its boundary. The boundary is partitioned into a Dirichlet boundary Γ_D and a Neumann boundary Γ_N, such that $\Gamma_D \cup \Gamma_N = \partial\Omega$ and $\Gamma_D \cap \Gamma_N = \emptyset$.
According to the project specifications, the strong formulation of the problem is defined as follows:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} - \nu\Delta\mathbf{u} + \nabla p = f & \text{in } \Omega, \\ \nabla \cdot \mathbf{u} = 0 & \text{in } \Omega, \\ \mathbf{u} = \mathbf{g} & \text{on } \Gamma_D \subset \partial\Omega, \\ \nu\nabla\mathbf{u}\,\mathbf{n} - p\mathbf{n} = \mathbf{h} & \text{on } \Gamma_N = \partial\Omega \setminus \Gamma_D, \\ \mathbf{u}(t=0) = \mathbf{u}_0 & \text{in } \Omega. \end{cases} \tag{2}$$

Here, $\mathbf{u}$ represents the fluid velocity, p is the pressure, ν is the kinematic viscosity, and f represents the external body forces

2.2. Weak Formulation

To derive the weak formulation, we must introduce the appropriate infinite-dimensional function spaces. Let $L^2(\Omega)$ be the space of square-integrable functions on Ω, and $H^1(\Omega)$ be the standard Sobolev space

$$V = \{\mathbf{v} \in [H^1(\Omega)]^d : \mathbf{v}|_{\Gamma_D} = \mathbf{g}\} \tag{3}$$

$$V_0 = \{\mathbf{v} \in [H^1(\Omega)]^d : \mathbf{v}|_{\Gamma_D} = \mathbf{0}\} \tag{4}$$

$$Q = L^2(\Omega) \tag{5}$$

We proceed by multiplying the momentum equation by a vector test function $\mathbf{v} \in V_0$ and the continuity equation by a scalar test function $q \in Q$, integrating both over the domain Ω
Applying the First Green's Identity (integration by parts) to the diffusive term and the pressure gradient, we obtain:

$$-\int_\Omega \nu\Delta\mathbf{u} \cdot \mathbf{v}\, d\Omega = \int_\Omega \nu\nabla\mathbf{u} : \nabla\mathbf{v}\, d\Omega - \int_{\partial\Omega} \nu(\nabla\mathbf{u} \cdot \mathbf{n}) \cdot \mathbf{v}\, d\Gamma \tag{6}$$

$$\int_\Omega \nabla p \cdot \mathbf{v}\, d\Omega = -\int_\Omega p(\nabla \cdot \mathbf{v})\, d\Omega + \int_{\partial\Omega} (p\mathbf{n}) \cdot \mathbf{v}\, d\Gamma \tag{7}$$

Summing the boundary integrals from both terms yields $\int_{\partial\Omega}(\nu\nabla\mathbf{u}\,\mathbf{n} - p\mathbf{n}) \cdot \mathbf{v}\, d\Gamma$
To express the problem concisely, we introduce the following continuous bilinear and trilinear forms:

$$a(\mathbf{u}, \mathbf{v}) = \int_\Omega \nu\nabla\mathbf{u} : \nabla\mathbf{v}\, d\Omega \tag{8}$$

$$b(\mathbf{v}, q) = -\int_\Omega q(\nabla \cdot \mathbf{v})\, d\Omega \tag{9}$$

$$c(\mathbf{w}; \mathbf{u}, \mathbf{v}) = \int_\Omega ((\mathbf{w} \cdot \nabla)\mathbf{u}) \cdot \mathbf{v}\, d\Omega \tag{10}$$

The linear functional incorporating external forces and Neumann boundary conditions is:

$$F(\mathbf{v}) = \int_\Omega f \cdot \mathbf{v}\, d\Omega + \int_{\Gamma_N} \mathbf{h} \cdot \mathbf{v}\, d\Gamma \tag{11}$$

Therefore, the continuous weak problem is stated as:
Find $\mathbf{u}(t) \in V$ and $p(t) \in Q$ such that for all $\mathbf{v} \in V_0$ and $q \in Q$:

$$\begin{cases} (\frac{\partial \mathbf{u}}{\partial t}, \mathbf{v}) + a(\mathbf{u}, \mathbf{v}) + c(\mathbf{u}; \mathbf{u}, \mathbf{v}) + b(\mathbf{v}, p) = F(\mathbf{v}) \\ b(\mathbf{u}, q) = 0 \end{cases} \tag{12}$$

where $(\cdot, \cdot)$ denotes the standard $L^2(\Omega)$ inner product

2.3. Spatial Discretization: Finite Element Method

To discretize the continuous weak problem in space, we introduce a finite element triangulation $\mathcal{T}_h$ of the domain Ω. The infinite-dimensional function spaces V and Q are replaced by their finite-dimensional discrete counterparts $V_h \subset V$ and $Q_h \subset Q$.
To ensure numerical stability and satisfy the Ladyzhenskaya-Babuška-Brezzi (inf-sup) condition, the solver employs Taylor-Hood finite element pairs
By expressing the discrete velocity $\mathbf{u}_h$ and pressure p_h fields as linear combinations of their respective basis functions, the semi-discrete problem can be rewritten as a system of Ordinary Differential Equations (ODEs):

$$\begin{cases} M\dot{\mathbf{U}} + A\mathbf{U} + C(\mathbf{U})\mathbf{U} + B^T\mathbf{P} = \mathbf{F} \\ B\mathbf{U} = \mathbf{0} \end{cases} \tag{13}$$

where $\mathbf{U}$ and $\mathbf{P}$ are the vectors containing the degrees of freedom for velocity and pressure, M is the fluid mass matrix, A is the viscous stiffness matrix, $C(\mathbf{U})$ is the non-linear convection matrix, and B is the divergence matrix

2.4. Temporal Discretization and Linearization

To obtain the fully discrete formulation, the time interval $(0, T]$ is partitioned into sub-intervals of constant width Δt, such that $t^n = n\Delta t$. The temporal derivative is approximated using the unconditionally stable, first-order implicit Backward Euler scheme, which is explicitly implemented in the solver's assembly routines

$$\frac{\partial \mathbf{u}}{\partial t} \approx \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} \tag{14}$$

A major computational challenge in solving the Navier-Stokes equations is the non-linear convective term $c(\mathbf{u}; \mathbf{u}, \mathbf{v})$. The solver utilizes a semi-implicit linearization strategy (Oseen-type linearization)

$$c(\mathbf{u}^{n+1}; \mathbf{u}^{n+1}, \mathbf{v}) \approx c(\mathbf{u}^n; \mathbf{u}^{n+1}, \mathbf{v}) \tag{15}$$

This approach decouples the non-linearity, reducing the problem to a single linear saddle-point system to be solved at each time increment

2.5. Temam's Stabilization

At the continuous level, the convective operator is skew-symmetric under the assumption that the velocity field is strictly divergence-free ($\nabla \cdot \mathbf{u} = 0$). However, in the standard Taylor-Hood finite element formulation, the divergence-free constraint is enforced only weakly. Consequently, the discrete advecting velocity $\mathbf{u}_h^n$ is not pointwise divergence-free, leading to a loss of skew-symmetry in the convective matrix $C(\mathbf{U}^n)$ which can cause numerical instabilities.
To prevent this loss of coercivity, the solver implements Temam's stabilization term

$$s(\mathbf{u}^n; \mathbf{u}^{n+1}, \mathbf{v}) = \frac{1}{2} \int_\Omega (\nabla \cdot \mathbf{u}^n)\mathbf{u}^{n+1} \cdot \mathbf{v}\, d\Omega \tag{16}$$

This mathematically restores the skew-symmetry of the discrete operator, guaranteeing that the convective term does not spuriously produce or dissipate kinetic energy, thus ensuring unconditional stability of the linearized scheme.

2.6. Fully Discrete Linear System

Combining the Backward Euler time-stepping, the semi-implicit linearization, and the Temam stabilization, the problem solved at each time step t^{n+1} is a linear algebraic system of the form:

$$\begin{bmatrix} \frac{1}{\Delta t}M + A + C_{stab}(\mathbf{U}^n) & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U}^{n+1} \\ \mathbf{P}^{n+1} \end{bmatrix} = \begin{bmatrix} \mathbf{G} \\ \mathbf{0} \end{bmatrix} \tag{17}$$

where $C_{stab}(\mathbf{U}^n)$ incorporates both the linearized convection and the stabilization term, and the right-hand side vector $\mathbf{G}$ incorporates the contribution from the previous time step $\frac{1}{\Delta t}M\mathbf{U}^n$ as well as any external and boundary forces

3. Preconditioners

In this section, we present the preconditioning strategies implemented in our solver, adapted from the parallel framework proposed by [1] for hemodynamic applications. For each method, we describe its exact formulation followed by its approximate variant. The goal of using these approximate forms is to reduce the time spent per iteration, achieving an optimal trade-off between iteration count and total wall-clock time.

3.1. SIMPLE preconditioner

The Semi-Implicit Method for Pressure Linked Equations has been introduced as an iterative method which first solves the momentum equation and then updates the pressure field and the velocity field to conserve the mass by using the continuity equation. The method can be reinterpreted as if it were associated with the following preconditioner,

$$P_{\text{SIMPLE}} = \begin{bmatrix} F & 0 \\ B & -\tilde{S} \end{bmatrix} \begin{bmatrix} I & D^{-1}B^T \\ 0 & \alpha I \end{bmatrix} \tag{18}$$

where D is the diagonal of F, $\alpha \in (0, 1]$ is a parameter that damps the pressure update, and $\tilde{S} = BD^{-1}B^T$. Its approximate form (aSIMPLE) replaces the exact inverses of the block operators with suitable approximations, denoted by a "hat" $(\hat{\cdot})$ over the operators. The resulting approximate preconditioner inverse is given by:

$$P^{-1}_{\text{aSIMPLE}} = \begin{bmatrix} D^{-1} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} I & -B^T \\ 0 & I \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & \frac{1}{\alpha} I \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & -\hat{\tilde{S}}^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ -B & I \end{bmatrix} \begin{bmatrix} \hat{F}^{-1} & 0 \\ 0 & I \end{bmatrix} \tag{19}$$

3.2. Yosida preconditioner

This preconditioner is inspired by the Yosida method, introduced by [2] for solving the incompressible Navier–Stokes equations. Its associated preconditioning matrix is given by:

$$P_{\text{Y}} = \begin{bmatrix} F & 0 \\ B & -\Delta t B M_u^{-1} B^T \end{bmatrix} \begin{bmatrix} I & F^{-1}B^T \\ 0 & I \end{bmatrix} \tag{20}$$

where $\Delta t B M_u^{-1} B^T$ is an approximation for the Schur complement S in the original method
The approximate Yosida preconditioner is derived using the same strategy employed by aSIMPLE in equation (19)

$$P^{-1}_{\text{aYosida}} = \begin{bmatrix} \hat{F}^{-1} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} I & -B^T \\ 0 & I \end{bmatrix} \begin{bmatrix} F & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & -\hat{S}^{-1} \end{bmatrix} \begin{bmatrix} I & 0 \\ -B & I \end{bmatrix} \begin{bmatrix} \hat{F}^{-1} & 0 \\ 0 & I \end{bmatrix}, \tag{21}$$

4. Drag and Lift Forces and Coefficients

In the context of laminar flow around a cylinder, the drag and lift forces are fundamental quantities used to characterize the aerodynamic behavior of the flow. These forces are computed as surface integrals over the cylinder's boundary, accounting for both viscous and pressure contributions.

4.1. Mathematical Definitions

For both 2D and 3D cases, the drag force F_D and lift force F_L are defined as:

$$F_D = \int_S \left(\rho \nu \frac{\partial v_t}{\partial n} n_y - P n_x \right) dS \tag{22}$$

$$F_L = -\int_S \left(\rho \nu \frac{\partial v_t}{\partial n} n_x + P n_y \right) dS \tag{23}$$

where: S is the surface of the cylinder, ρ is the fluid density (set to $1.0\,\text{kg/m}^3$ in the benchmark), ν is the kinematic viscosity ($10^{-3}\,\text{m}^2/\text{s}$), P is the pressure, $\mathbf{n} = (n_x, n_y, n_z)$ is the outward unit normal vector on S, v_t is the tangential velocity on S, $\mathbf{t}$ is the tangent vector, defined as $\mathbf{t} = (n_y, -n_x)$ for 2D cases and $\mathbf{t} = (n_y, -n_x, 0)$ for 3D cases.

4.2. Drag and Lift Coefficients

The drag coefficient c_D and lift coefficient c_L normalize the forces with respect to the dynamic pressure and a reference area. Their definitions differ between 2D and 3D configurations:

- 2D Cases:

$$c_D = \frac{2F_D}{\rho \overline{U}^2 D}, \quad c_L = \frac{2F_L}{\rho \overline{U}^2 D} \tag{24}$$

  where $D = 0.1\,\text{m}$ is the cylinder diameter, and $\overline{U}$ is the mean inflow velocity.
- 3D Cases:

$$c_D = \frac{2F_D}{\rho \overline{U}^2 DH}, \quad c_L = \frac{2F_L}{\rho \overline{U}^2 DH} \tag{25}$$

  where $H = 0.41\,\text{m}$ is the channel height (or width), and D remains the cylinder diameter (or side length for square cross-sections).

4.3. Physical Interpretation

The drag coefficient c_D quantifies the resistance experienced by the cylinder in the direction of the flow, while the lift coefficient c_L captures the transverse force due to asymmetric pressure distribution or vortex shedding. These coefficients are critical for validating numerical methods, as they provide a direct comparison with experimental data and serve as key metrics for assessing the accuracy and reliability of the simulations.

5. Implementation

The solver is implemented in C++ using the **deal.II** library. To support both **2D** and **3D** simulations, the main logic is encapsulated in a templated *NavierStokes* class. The source code is available at [3].

5.1. Mesh

The computational meshes are generated using **Gmsh** via its Python API, following the geometry and spatial discretization guidelines specified in the benchmark [4]. To balance numerical accuracy with computational efficiency, spatially non-uniform, unstructured element topologies are constructed for both two- and three-dimensional configurations, using triangular elements in 2D and tetrahedral elements in 3D.
Local spatial refinement (h-adaptation) is systematically concentrated along the cylinder boundary and within the immediate downstream wake region. High element density in these critical zones ensures precise resolution of steep velocity gradients, boundary layer shear dynamics, and unsteady vortex shedding. This localized resolution is essential for accurately evaluating integrated hydrodynamic parameters—specifically the drag coefficient C_D, lift coefficient C_L, and point-wise pressure differentials Δp.
Moving away from the obstacle toward the outer domain boundaries (inlet, outlet, and lateral walls), the mesh transitions smoothly through a geometric growth factor to a coarser element distribution. This graded discretization minimizes the total number of degrees of freedom (DoFs), thereby reducing memory overhead and floating-point computation time for the iterative linear solvers without sacrificing global solution convergence in high-gradient regions.

5.2. Navier-Stokes Solver Implementation Overview

This subsection summarizes the C++ implementation of a parallel finite element solver for the incompressible Navier-Stokes equations using the `deal.II` library. The solver, defined in `NavierStokes.hpp` and implemented in `NavierStokes.cpp`, supports 2D and 3D configurations, steady and unsteady inflow regimes, and MPI-based parallel computation.

5.2.1 Governing Equations and Discretization

The solver addresses the incompressible Navier-Stokes equations with constant fluid properties:

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} - \nu \nabla^2 \mathbf{u} + \nabla p = \mathbf{f}, \tag{26}$$

$$\nabla \cdot \mathbf{u} = 0, \tag{27}$$

where $\nu = 10^{-3}\,\mathrm{m}^2/\mathrm{s}$ is the kinematic viscosity, $\rho = 1.0\,\mathrm{kg}/\mathrm{m}^3$ is the density, $\mathbf{u}$ is the velocity, and p is the pressure. The time derivative is discretized with the backward Euler method, which treats the diffusive part of the operator implicitly and unconditionally stably, while space is discretized with Taylor-Hood P_2-P_1 elements, an inf-sup stable pairing that avoids spurious pressure oscillations without requiring additional pressure stabilization. As we have described before, to improve the discrete conservation properties of the convective term, a Temam stabilization term $0.5\int(\nabla\cdot\mathbf{u})\mathbf{u}\cdot\mathbf{v}\,d\Omega$ is added to the weak formulation, compensating for the fact that the discrete velocity field is not exactly divergence-free.
The weak form includes the viscous term $\nu\int\nabla\mathbf{u}:\nabla\mathbf{v}\,d\Omega$, the convective term $\int(\mathbf{u}\cdot\nabla)\mathbf{u}\cdot\mathbf{v}\,d\Omega$ together with the Temam correction above, the mass matrix term $\int\mathbf{u}^{n+1}\cdot\mathbf{v}/\Delta t\,d\Omega$ arising from the implicit time derivative, and the pressure coupling terms $-\int p\nabla\cdot\mathbf{v}\,d\Omega$ and $\int q\nabla\cdot\mathbf{u}\,d\Omega$, which together enforce incompressibility in the coupled velocity-pressure system.

5.2.2 Architecture and Key Methods

The solver is implemented as a templated class, `NavierStokes<dim>`, parametrized on the spatial dimension and organized around a small set of methods that mirror the natural stages of a time-dependent finite element simulation.
The `setup()` method performs all one-time initialization. It reads the computational mesh from the corresponding `.msh` file and distributes it across MPI ranks, so that every process subsequently operates only on its locally owned and ghost cells. It instantiates the mixed finite element space, coupling a P_2 simplex element for each velocity component with a P_1 simplex element for pressure, and distributes the degrees of freedom with a component-wise renumbering that groups all velocity unknowns into a first block and all pressure unknowns into a second. This renumbering is what allows the rest of the solver to operate directly on block matrices and block vectors (`TrilinosWrappers::BlockSparseMatrix`, `TrilinosWrappers::MPI::BlockVector`). Within `setup()`, the block sparsity patterns for the coupled system matrix and for the auxiliary pressure-space matrices used by the preconditioners are built, every matrix and vector is reinitialized against the appropriate pattern, and the smallest cell diameter in the mesh is computed once and stored, since it is later needed to monitor the advective Courant number during time-stepping.
`assemble()` builds, once, every operator that does not change from one time step to the next: the velocity mass matrix, associated with the implicit backward Euler discretization of the time derivative; the viscous stiffness matrix, associated with the term $\nu\int\nabla\mathbf{u}:\nabla\mathbf{v}\,d\Omega$; and the two pressure-velocity coupling blocks corresponding to $-\int p\,\nabla\cdot\mathbf{v}\,d\Omega$ and $\int q\,\nabla\cdot\mathbf{u}\,d\Omega$. It also assembles the pressure mass matrix used by the Schur-complement approximation in the SIMPLE- and Yosida-type preconditioners. Finally, `assemble()` imposes the Dirichlet velocity conditions at the inlet, on the walls, and on the obstacle.
Because only the convective term and the right-hand side depend on the solution computed at the previous time step, `assemble_time_step()` avoids rebuilding the constant operators produced by `assemble()` and instead reassembles, at every step, the convection matrix, linearized around the velocity field from the previous time step together with its Temam stabilization contribution, and the mass-matrix contribution to the right-hand side that realizes the backward Euler discretization of the time derivative, $\mathbf{M}\mathbf{u}^n/\Delta t$. The Dirichlet boundary conditions on the velocity are reapplied after this partial reassembly, since they must remain consistent with the updated right-hand side.
The time loop itself is orchestrated by `run()`: at every step it advances the current simulation time, calls `assemble_time_step()` to bring the linear system up to date, invokes `solve_time_step()` to obtain the new velocity and pressure fields, and then computes and records the diagnostics used throughout the rest of this report, namely the advective CFL number $\|\mathbf{u}\|_\infty\Delta t/h_{\min}$, the drag and lift forces and their non-dimensional coefficients through `compute_forces()`, and, close to the final time, the pressure difference between two reference points across the obstacle through `compute_pressure_difference()`, before writing the solution fields to disk through `output()`.
`solve_time_step()` is responsible for solving the coupled velocity-pressure system at every step. It constructs and initializes the preconditioner selected for the run (§5.2.3), configures a `SolverGMRES` with a residual tolerance set relative to the norm of the right-hand side, and solves the block system. The wall time spent initializing the preconditioner and the wall time spent in the GMRES solve are both recorded at every step, which makes it possible to compare the different preconditioning strategies not only by iteration count but by their actual computational cost.
`compute_forces()` evaluates the hydrodynamic forces exerted by the fluid on the obstacle by integrating the Cauchy stress tensor, $\boldsymbol{\sigma} = -p\,\mathbf{I} + \nu\,(\nabla\mathbf{u} + \nabla\mathbf{u}^\top)$, over the obstacle boundary using `FEFaceValues` restricted to the faces marked with `id_obstacle`. The resulting force is decomposed into drag and lift components, which are then non-dimensionalized into c_D and c_L using the reference dynamic pressure and the obstacle diameter, following the convention of the benchmark under consideration. `compute_pressure_difference()` complements this by evaluating the pressure at two fixed points immediately upstream and downstream of the obstacle and reporting their difference.

Finally, `output()` writes the velocity and pressure fields, together with the MPI subdomain identifier of each cell, to `.vtu`/`.pvtu` files at regular intervals during the simulation, so that the time evolution of the flow can be inspected in ParaView.

5.2.3 Preconditioners

The solver supports multiple preconditioning strategies for the block system, all sharing the same outer GMRES iteration and differing only in how they approximate the inverse of the saddle-point matrix. `BLOCK_IDENTITY` applies no preconditioning at all and serves as a baseline against which the benefit of the other strategies is measured. `SIMPLE` and its cheaper variant `APPROX_SIMPLE` build a block-triangular preconditioner around an algebraic approximation of the Schur complement, controlled by a relaxation factor set to $\alpha = 0.5$ in all runs. `YOSIDA` and `APPROX_YOSIDA` instead approximate the Schur complement through an operator splitting that makes explicit use of the velocity mass matrix, in the spirit of the Yosida method for the incompressible Navier-Stokes equations.

5.2.4 Boundary Conditions and Input Parameters

Boundary conditions are prescribed according to the role of each portion of the domain boundary. At the inlet (`id_inlet = 1`) a parabolic velocity profile is imposed,

$$\text{2D: } U(0,y) = \frac{4U_m y(H-y)}{H^2}, \tag{28}$$

$$\text{3D: } U(0,y,z) = \frac{16U_m yz(H-y)(H-z)}{H^4}, \tag{29}$$

with the unsteady inflow regime modulating this profile in time as $U(0,y,t) = U(0,y)\sin(\pi t/8)$ to model a periodically driven inlet. On the walls (`id_walls = 3`) and on the obstacle (`id_obstacle = 4`) a no-slip condition $\mathbf{u} = \mathbf{0}$ is enforced. At the outlet (`id_outlet = 2`) no condition is imposed explicitly: the boundary integral $\int_\Gamma (\nu \nabla \mathbf{u} \cdot \mathbf{n} - p\,\mathbf{n}) \cdot \mathbf{v}\, d\Gamma$ that arises from the weak formulation is simply omitted, which corresponds to the classical do-nothing condition, i.e. a vanishing total traction (viscous plus pressure) rather than a literal pointwise Dirichlet condition on the pressure alone, although the two coincide approximately when the flow near the outlet is close to fully developed.

5.2.5 Output and Diagnostics

The solver produces several complementary outputs over the course of a run. The solution fields, velocity and pressure, are written in `.vtu` format together with the MPI partitioning information, for visualization in ParaView. The time histories of the drag and lift forces and of their non-dimensional coefficients are accumulated in `vec_drag_force`, `vec_lift_force`, `vec_drag_coeff`, and `vec_lift_coeff`. The pressure difference across the obstacle is recorded near the final time step. The advective Courant number, $CFL = \|\mathbf{u}\|_\infty \cdot \Delta t / h_{min}$, is tracked at every time step as a diagnostic of the temporal accuracy of the semi-implicit treatment of convection. Finally, the wall time spent, respectively, initializing the preconditioner and performing the GMRES solve is recorded in `time_preconditioning` and `time_solve`, allowing the different preconditioning strategies to be compared on a genuinely comparable computational basis.

6. Results

In the following section, we show the results achieved with our solution on the different test cases presented in the benchmark [4].

6.1. 2D-1 Test Case

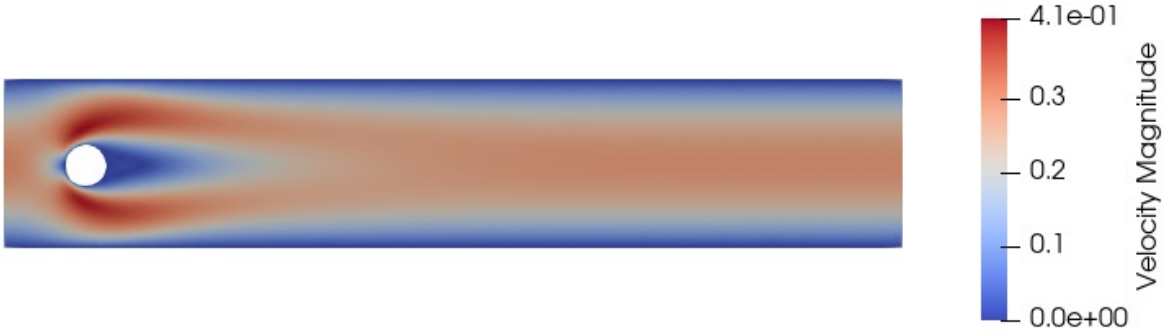
For this test case, the inflow condition is given by

$$U(0,y) = \frac{4U_m y(H-y)}{H^2}, \quad V = 0,$$

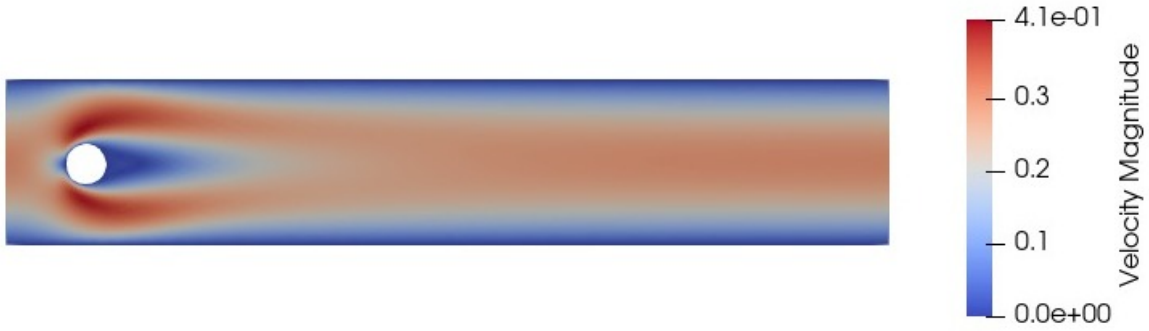
with $U_m = 0.3\,\mathrm{m/s}$, yielding a Reynolds number of $\mathrm{Re} = 20$. The configuration file for this test case is located at `test/config_2D1.txt`. We solve the system over the time interval $T = 20.0\,\mathrm{s}$ using a time step of $\Delta t = 0.1\,\mathrm{s}$. After 200 iterations, the simulation finishes with an average execution time of $0.89\,\mathrm{s} \pm 0.86\,\mathrm{s}$ per iteration.

Since Re = 20, the flow is expected to reach a steady state. This is confirmed by the asymptotic behavior of the coefficients and the velocity field for $t > 5.0\,\mathrm{s}$, where the solution transitions toward a steady configuration (Figure 1), and by the drag and lift coefficient trends stabilizing at 3.5 and near 0, respectively.

(a) Initial transient development of the velocity field at $t = 1.0\,\mathrm{s}$.



(b) Transition toward a steady configuration at $t = 5.0\,\mathrm{s}$.



(c) Fully developed steady state velocity field at $t = 20.0\,\mathrm{s}$.

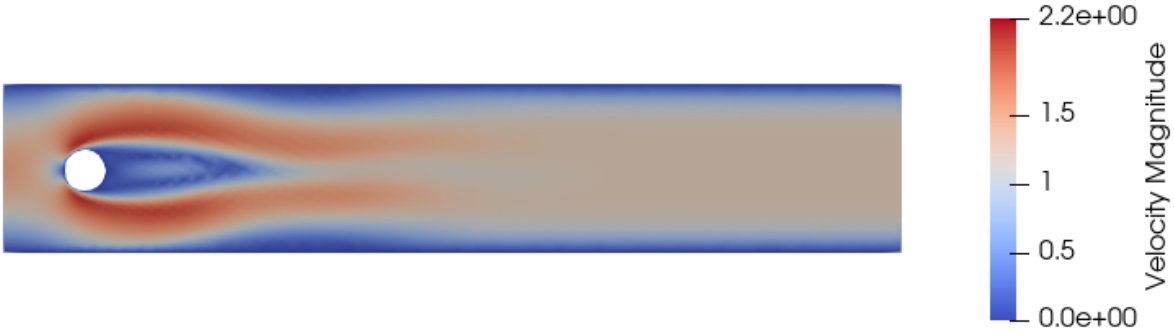
Figure 1: Time evolution 2D-1 of the velocity field for Re = 20.

6.2. 2D-2 Test Case: Flow Around a Cylinder

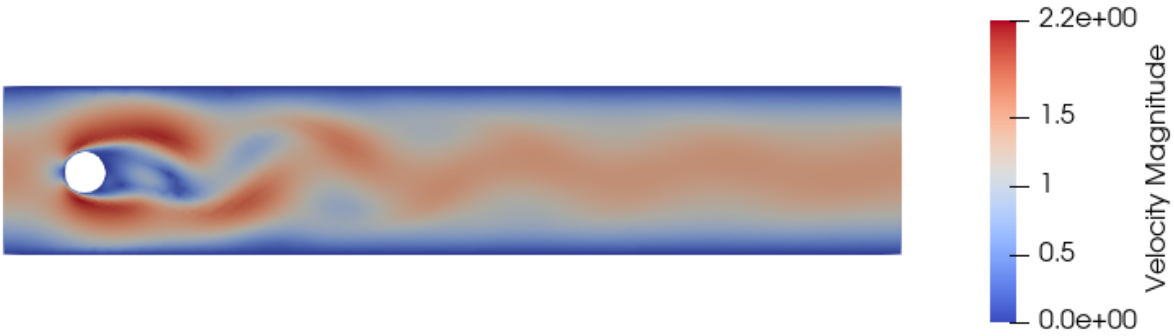
For the 2D-2 benchmark test case, the inflow condition is given by

$$U(0, y) = \frac{4U_m y(H - y)}{H^2}, \quad V = 0,$$

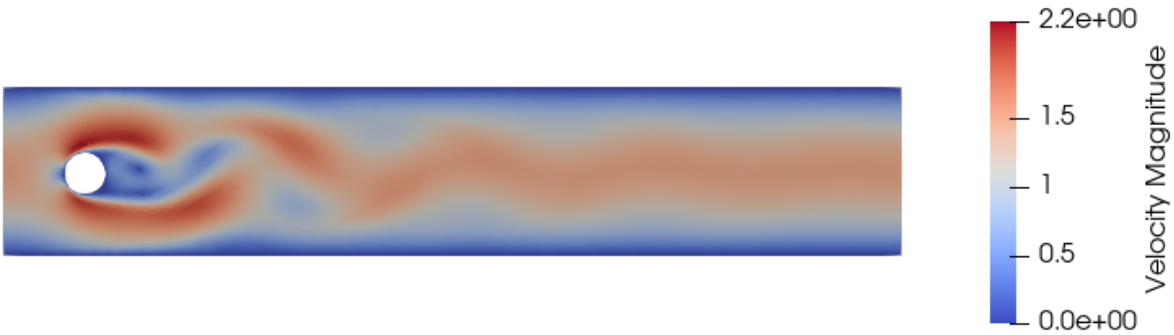
with a maximum inflow velocity of $U_m = 1.5\,\mathrm{m/s}$, corresponding to a Reynolds number of Re = 100. The configuration file for this setup is located at `test/config_2D2.txt`.
We simulate the system over $T = 30.0\,\mathrm{s}$ with a refined time step of $\Delta t = 0.01\,\mathrm{s}$. The computational cost is 0.79 s per iteration (std 0.076 s), with a total wall time of 2375 s.
Because Re = 100 exceeds the critical Reynolds number, the flow observed in the 2D-1 case becomes unstable, as shown in Figure 2. The drag coefficient stabilizes over time (Figure 5), while the lift force develops periodically, forming a periodic wave. (Figure 4)



(a) Initial transient development of the velocity field at $t = 1.0\,\mathrm{s}$.



(b) Transition toward a steady configuration at $t = 3.5\,\mathrm{s}$.



(c) Fully developed steady state velocity field at $t = 30.0\,\mathrm{s}$.

Figure 2: Time evolution 2D-2 of the velocity field for $\mathrm{Re} = 100$.

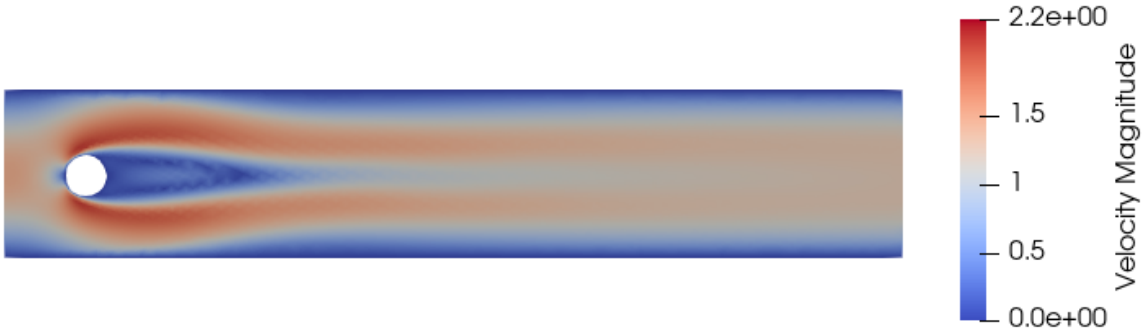
6.3. 2D-3 Test Case: Unsteady Flow

The inflow condition is given by

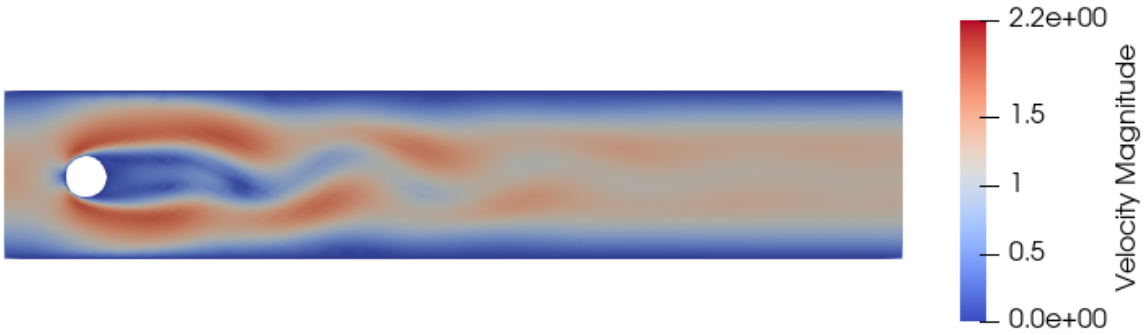
$$U(0, y, t) = \frac{4U_m y(H - y)\sin(\pi t/8)}{H^2}, \quad V = 0,$$

with $U_m = 1.5\,\mathrm{m/s}$. The time interval is $0 \leq t \leq 8\,\mathrm{s}$, using a time step of $\Delta t = 0.005\,\mathrm{s}$. This unsteady case yields a Reynolds number varying over $0 \leq \mathrm{Re}(t) \leq 100$. The configuration file for this setup is located at `test/config_2D3.txt`. The simulation runs for 1600 iterations, with an average solving time per time step of 0.4 s.

(a) Velocity for test case 2D-3 at $t = 1.0\,\mathrm{s}$.



(b) Velocity for test case 2D-3 at $t = 3.5\,\mathrm{s}$.



(c) Velocity for test case 2D-3 at $t = 5.0\,\mathrm{s}$.

Figure 3: Time evolution of the velocity field with sinusoidal input in 2D.

In Figure 4, we compare the lift force plots across the unsteady cases. For the last case, the lift coefficient value is around zero during the initial acceleration phase. Once the Reynolds number approaches $\mathrm{Re} \approx 100$, periodic instability begins, causing the amplitude of the lift oscillations to grow rapidly and peak between $t = 5.0\,\mathrm{s}$ and $t = 6.0\,\mathrm{s}$, before decreasing and stabilizing as the velocity approaches zero toward $t = 8.0\,\mathrm{s}$.
On the other hand, the drag coefficient follows the macroscopic acceleration and deceleration of the flow. It rises from zero, reaching a peak of approximately 3.0 around $t = 4.0\,\mathrm{s}$ corresponding to the maximum flow inertia, and then gradually decreases during the decelerating phase while exhibiting small oscillations from vortex shedding (Figure 6).

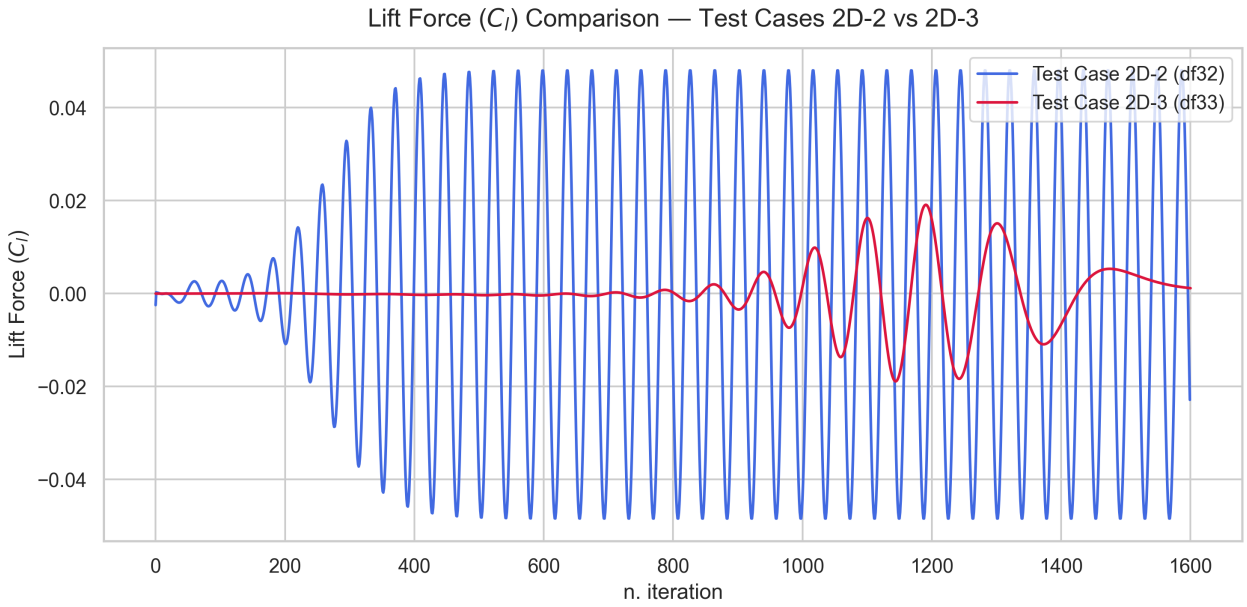


Figure 4: Lift coefficient over time for the 2D unsteady test cases.

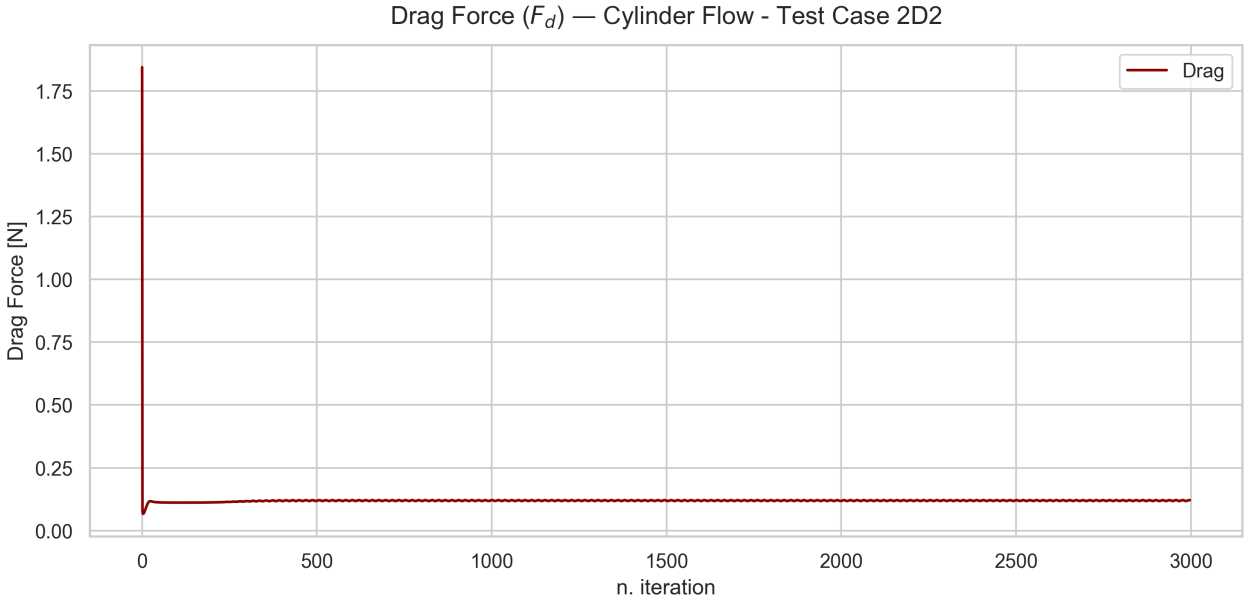


Figure 5: Drag force coefficient over time for the 2D-2 test case.

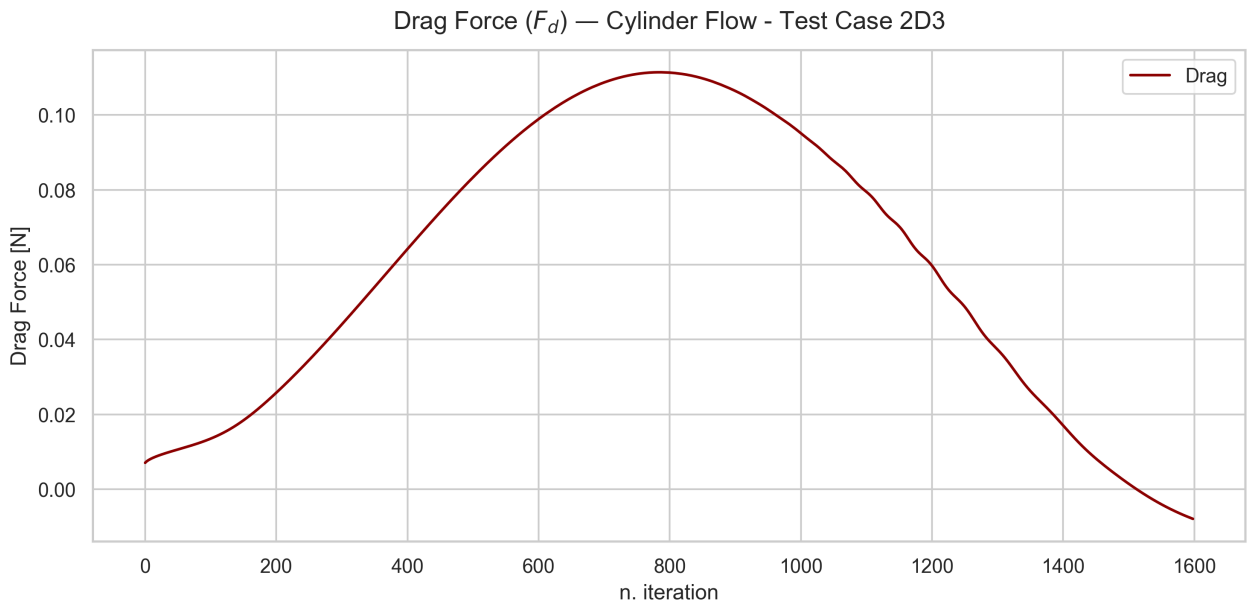


Figure 6: Drag force coefficient over time for the 2D-3 test case.

6.4. 3D-1 Test Case

The inflow condition is given by

$$U(0, y, z) = \frac{16U_m yz(H - y)(H - z)}{H^4}, \quad V = W = 0,$$

with $U_m = 0.45\,\text{m/s}$, yielding a Reynolds number of $Re = 20$.
We simulate the system over $T = 20.0s$ with a time step of $\Delta t = 0.1s$. The simulations consists of 200 iterations with an average solving time per iteration of 0.68 seconds. Towards the end, the solution reaches a final steady configuration, as shown in Figure 7b.

6.5. 3D-2 Test Case

The inflow condition is given by

$$U(0, y, z, t) = \frac{16U_m yz(H - y)(H - z)}{H^4}, \quad V = W = 0,$$

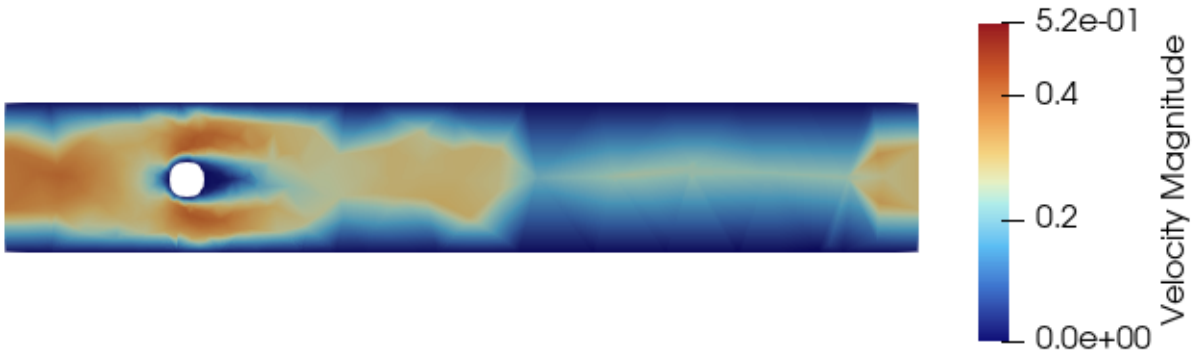
with $U_m = 2.25\,\text{m/s}$, yielding a Reynolds number of $Re = 100$.
For this test case, the simulation parameters are a total time $T = 30.0s$, a time step $t = 0.01s$, and a total execution time of $798s$.

6.6. 3D-3 Test Case

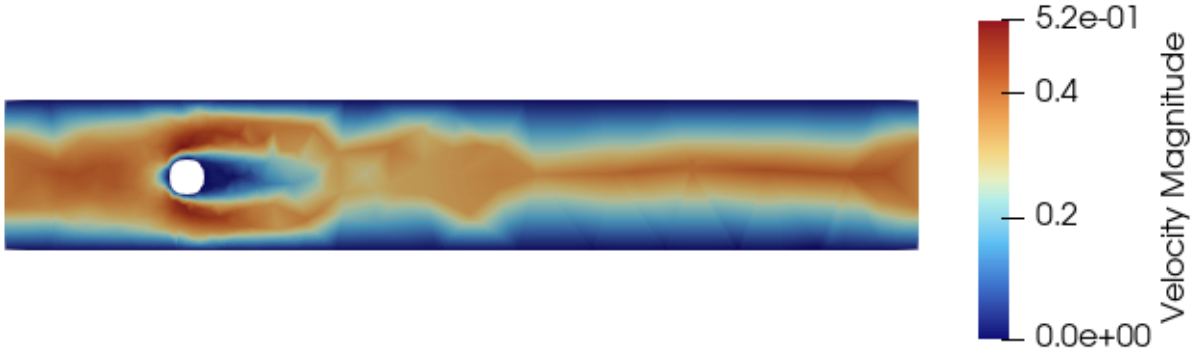
The inflow condition is given by

$$U(0, y, z, t) = \frac{16U_m yz(H - y)(H - z)\sin(\pi t/8)}{H^4}, \quad V = W = 0,$$

with $U_m = 2.25\,\text{m/s}$. The time interval is $0 \leq t \leq 8\,\text{s}$.
The simulation parameters are a total time $T = 8.0s$, a time step $t = 0.005s$, and a total execution time of $828s$.
At $t = 0.1$ s, the overall velocity magnitude is extremely low and the cylinder does not perturb the flow (Figure 9a). Then, at $t = 3.5$ s, the wake behind the cylinder elongates and the velocity reaches its peak (Figure 9b).
For the unsteady 3D test cases, we plot the lift coefficient over time, whereas the drag behavior remains similar to the corresponding 2D cases (Figure 10).

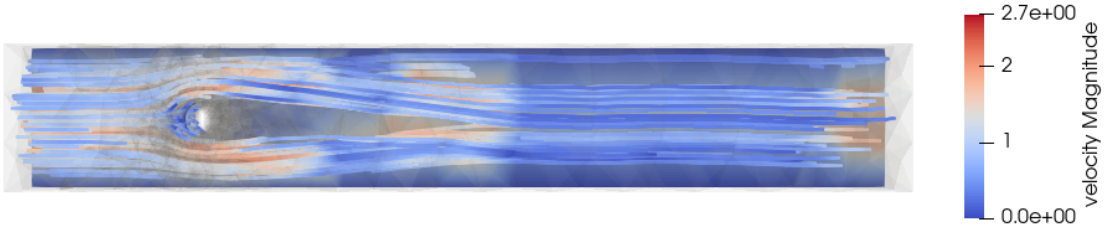


(a) Initial transient development for of the velocity field at $t = 0.1\,\mathrm{s}$.

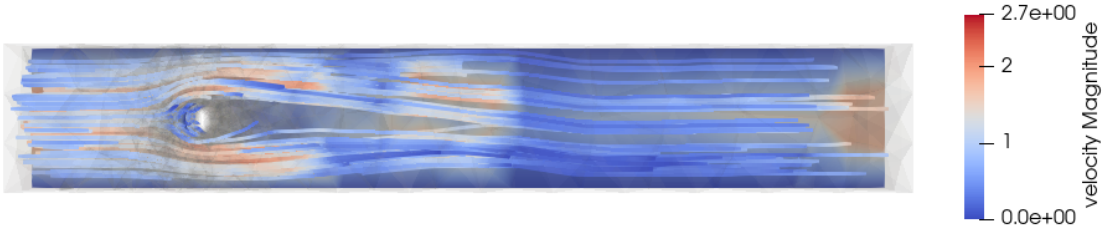


(b) Transition toward a steady configuration at $t = 20.0\,\mathrm{s}$.

Figure 7: Time evolution 3D-1 of the velocity field for $\mathrm{Re} = 20$.

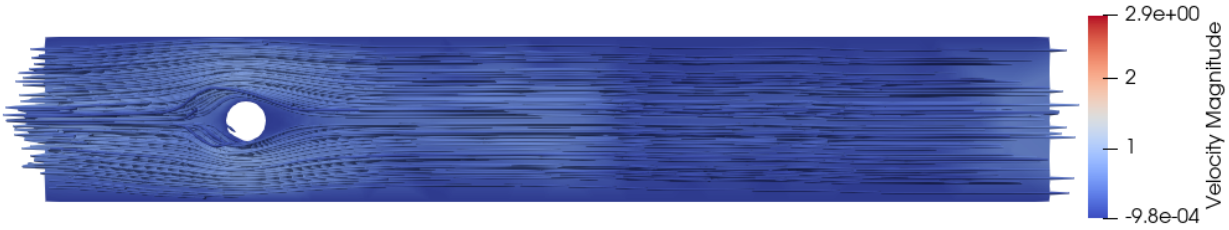


(a) Velocity flow for test case 3D-2 at $t = 0.1\,\mathrm{s}$.

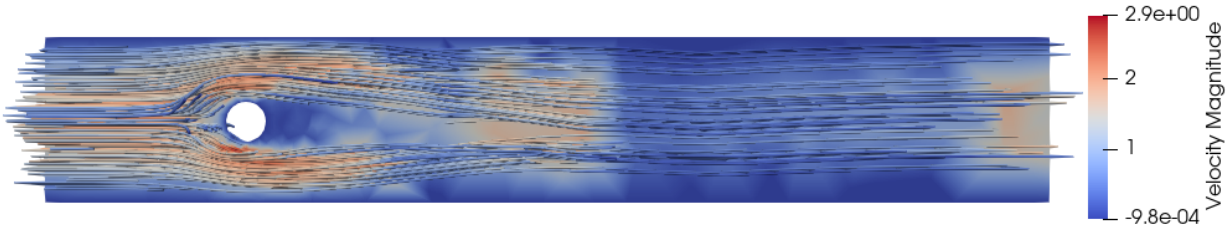


(b) Velocity flow for test case 3D-2 at $t = 30.0\,\mathrm{s}$.

Figure 8: Time evolution 3D-2 of the velocity field for $\mathrm{Re} = 100$.



(a) Velocity flow for test case 3D-3 at $t = 0.1\,\mathrm{s}$.



(b) Velocity flow for test case 3D-3 at $t = 3.5\,\mathrm{s}$.

Figure 9: Time evolution of the velocity field with sinusoidal input in 3D.

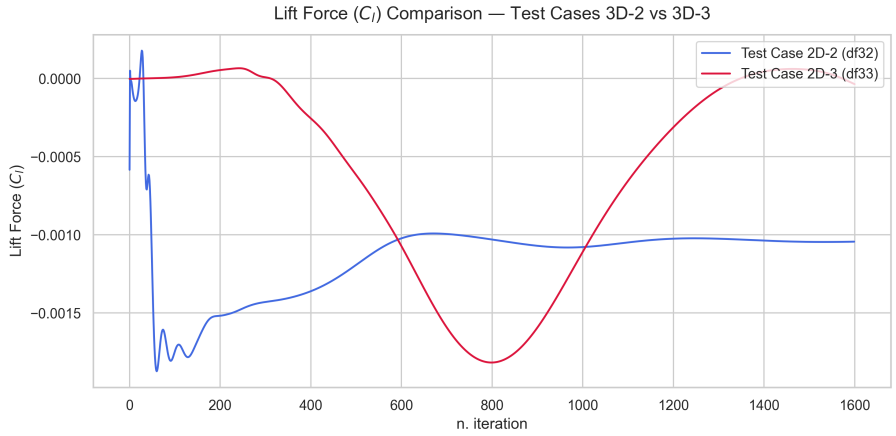


Figure 10: Comparison of the lift coefficient (C_l) over time for the 3D unsteady test cases.

6.7. Strong Scaling and Preconditioner Evaluation

In this section, we present the performance metrics evaluated on our benchmark. We execute test case 2D-3 on the MOX cluster for $T = 1.0\,\text{s}$ using various preconditioners across $n \in \{4, 8, 16, 32\}$ cores on a fine mesh.
By keeping the problem size fixed while increasing the core count (strong scaling analysis), we observe the following key trends:

- As shown in Figure 11, all preconditioners exhibit similar initialization times, which decrease predictably as the core count doubles.
- Approximate preconditioners require a higher number of GMRES iterations (Figure 12), and only the aSIMPLE method achieves the lowest solving time, making it the most optimal choice for running the simulations (Figure 13).

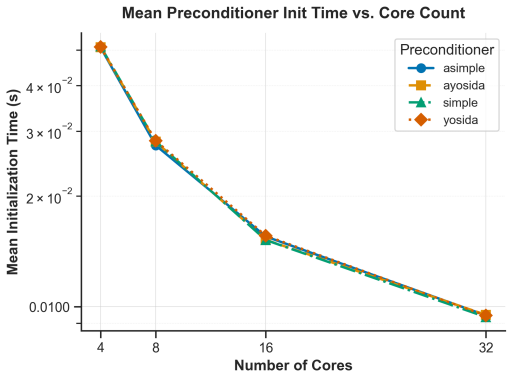


Figure 11: Initialization time for preconditioners with an increasing number of cores.

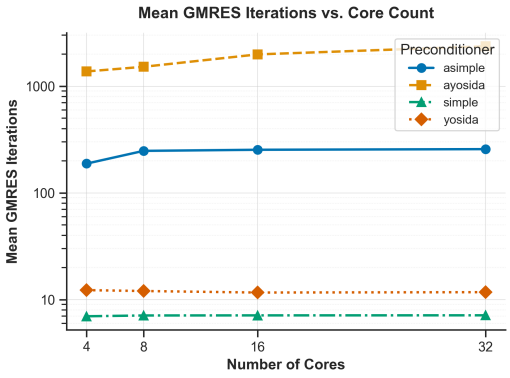


Figure 12: GMRES iterations for preconditioners with an increasing number of cores.

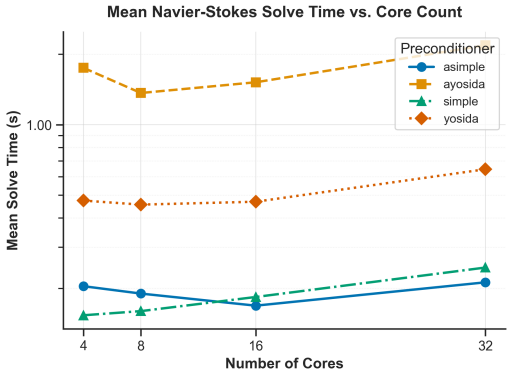


Figure 13: Mean solving time for preconditioners with an increasing number of cores.

7. Conclusions

This project successfully developed and implemented a computational system for simulating fluid flow around a cylindrical body, grounded in the scientific references outlined in the cited paper. In two-dimensional scenarios, the simulation produced accurate and reliable results for both steady-state flows and vortex-dominated regimes, consistently aligning with theoretical and empirical benchmarks. While the extension to three-dimensional simulations yielded less definitive outcomes, the work included a comprehensive preconditioning benchmark executed on the MOX cluster, providing a foundation for future refinements and validations.

8. Bibliography and citations

References

[1] G. Grandperrin S. Deparis and A. Quarteroni. Parallel preconditioners for the unsteady navier–stokes equations and applications to hemodynamics simulations. *Computers Fluids*, 2014.

[2] F. Saleri A. Quarteroni and A. Veneziani. Analysis of the Yosida method for the incompressible Navier–Stokes equations. *Journal de Mathématiques Pures et Appliquées*, 78(5):473–503, 1999.

[3] Daniele Cursano, Marco Melzi, and Lorenzo Terzi. navier-stokes-nmpde: Navier–Stokes equations for laminar flow around a cylinder. `https://github.com/marcomelzi/navier-stokes-nmpde`, 2025.

[4] Michael Schäfer, Stefan Turek, Franz Durst, Egon Krause, and Rolf Rannacher. Benchmark computations of laminar flow around a cylinder. In Ernst Heinrich Hirschel, editor, *Flow Simulation with High-Performance Computers II*, volume 52 of *Notes on Numerical Fluid Mechanics*, pages 547–566. Vieweg+Teubner Verlag, 1996.